Eulalia Integration Scheme Technical Document V2

Language: English
Status: Technical proof of concept (PoC)
Scope: Integration documentation, not production readiness or electoral certification

1. Purpose and Scope

This document updates the Eulalia integration scheme to include the standalone `eulalia-backend-cardano` service. It describes how the mobile app, web frontend, .NET backend, Identus infrastructure, PostgreSQL, and Cardano Preview transaction PoC fit together.

The scope is limited to the code, READMEs, and evidence available in the Eulalia repositories. It does not claim a production deployment, election certification, custody approval, or a successful Cardano Preview end-to-end transaction.

2. Version Change

V2 adds the Cardano Preview backend as a separate TypeScript/Fastify component. It accepts privacy-safe affiliation and cancellation event commitments, builds bounded metadata, signs with an institutional demo wallet held by the server environment, submits through Blockfrost, and stores demo status in a local JSON file.

3. Component Map

```
Citizen
  |
  v
eulalia-app (Flutter) ----\
                           > eulalia-backend (.NET 8) ---> PostgreSQL
eulalia-frontend (React) -/             |
                                        +--> Identus Cloud Agent ---> Prism/Cardano support

Affiliation or cancellation event
  |
  v
eulalia-backend-cardano (TypeScript/Fastify) ---> Blockfrost ---> Cardano Preview
              |
              +--> local JSON transaction state
```

Component	Technology	Responsibility	Interface or dependency
`eulalia-app`	Flutter/Dart	Citizen onboarding, wallet/SSI interaction, biometric flow, and affiliation UI	Calls the .NET API; default base URL is `http://localhost:5219`
`eulalia-frontend`	React, Vite, TypeScript	Web registration, authentication, organization, affiliation, and status views	Calls the .NET API through `VITE_API_BASE_URL`
`eulalia-backend`	.NET 8 ASP.NET Core/C#	REST API, JWT authentication, citizen and affiliation rules, SSI orchestration, webhook handling	PostgreSQL and Identus Cloud Agent; Swagger at the documented local URL
Identus/Cloud Agent	Hyperledger Identus 1.39,	DIDComm connections, credential offers, agent	Cloud Agent through `/cloud-agent`; PostgreSQL Docker instance supports the stack

Component	Technology	Responsibility	Interface or dependency
	Docker, Prism, Vault	persistence, and SSI infrastructure	
PostgreSQL	PostgreSQL 13+	Durable application and agent data in the existing architecture	Local application database and Identus Docker database are documented separately
`eulalia-backend-cardano`	Node.js 20+, TypeScript, Fastify, Lucid Evolution	Validates and records affiliation/cancellation event commitments on Preview	HTTP API on `127.0.0.1:3000`; Blockfrost Preview provider
Cardano Preview/Blockfrost	Cardano Preview and Blockfrost API	Network/provider access, transaction submission, and transaction status lookup	`https://cardano-preview.blockfrost.io/api/v0` by default

Repositories: mobile, frontend, .NET backend, Identus, and Cardano backend.

4. SSI and Identity Flow

1. The citizen registers and authenticates through the mobile app or frontend using the .NET API.
2. The .NET backend creates a DIDComm invitation through the Identus Cloud Agent.
3. The app presents the invitation URL as a QR code for a wallet.
4. The wallet accepts the connection; Identus sends a webhook event to the .NET backend.
5. The backend creates a credential offer, stores the holder and credential identifiers, and exposes the consolidated SSI status.
6. Affiliation is allowed only after the documented credential and biometric gates are satisfied.
7. The separate Cardano service receives a privacy-safe affiliation or cancellation event when that event is to be recorded on Preview.

The SSI identity flow and Cardano event record are separate boundaries: Cardano metadata is not the SSI credential store and does not replace the application database.

5. Cardano Affiliation and Cancellation Flow

The Cardano service exposes these authenticated operations:

Method	Endpoint	Purpose
GET	`/health`	Public service health and `Preview` network indication
POST	`/api/transactions/affiliations`	Record an affiliation event
POST	`/api/transactions/cancellations`	Record a cancellation event
GET	`/api/transactions/:eventId`	Read stored transaction status
POST	`/api/transactions/:eventId/refresh`	Ask Blockfrost for current status and persist it

The authenticated endpoints require `Authorization: Bearer <INTERNAL_API_TOKEN>`. Input is limited to `eventId`, `affiliationId`, `organizationId`, and `recordHash`. Identifiers are bounded, unknown fields are rejected, and `recordHash` must be a 64-character hexadecimal SHA-256 commitment.

The service persists `pending` before submission. It builds bounded metadata, signs and submits through Lucid and Blockfrost, then persists `submitted` with `txHash`. Provider errors persist `failed` with the safe code `BLOCKCHAIN_SUBMISSION_FAILED`. The supported status model also includes `confirmed` and `not_found` for status lookup.

Repeated requests are idempotent by `eventId`: an existing record is returned and a second transaction is not submitted. Concurrent requests for the same event share the in-flight operation. The refresh endpoint updates a submitted record from Blockfrost; terminal `confirmed` and `failed` records are not refreshed.

6. Privacy Boundary

The Cardano boundary accepts commitments and limited event metadata only. The documented metadata fields are `eventType`, `eventId`, `affiliationId`, `organizationId`, `recordHash`, `schemaVersion`, and `timestamp`, with a 1,600-byte safety bound and a configured metadata label.

No cedula, name, biometric data, raw citizen record, credential content, or other PII belongs on-chain. A SHA-256 commitment can support later integrity comparison only when the corresponding private record remains under controlled application storage. The CNE private key is server-side configuration and must not be returned or logged.

7. Configuration and Execution

Existing identity stack

Follow the repository READMEs for eulalia-identus, eulalia-backend, eulalia-frontend, and eulalia-app. The documented local services include the .NET API, PostgreSQL, Identus Cloud Agent, and the web/mobile clients.

Cardano PoC

The Cardano README documents Node.js 20+, a Blockfrost Preview project ID, a funded Preview wallet, and the following non-secret workflow:

```
npm install
npm run typecheck
npm test
npm run build
npm start
```

Configure `.env` from `.env.example` without committing secrets. The required values are `BLOCKFROST_PROJECT_ID`, `CNE_ADDRESS`, `CNE_PRIVATE_KEY`, and `INTERNAL_API_TOKEN`; optional values include `PORT`, `HOST`, `CARDANO_NETWORK` (must be `Preview`), `METADATA_LABEL`, `METADATA_SCHEMA_VERSION`, and `DATA_FILE`. Never copy secret values into this document.

8. Evidence and Limitations

- The current English and Spanish `Test Report` PDFs under `final-report/` are the baseline and reconciliation source for metrics.
- The historical 98.0% aggregate is not a current approval result.
- Existing evidence shows low backend coverage in the referenced report, approximately 12.4% line coverage and 25% branch coverage.
- Historical backend and frontend failures or environment-dependent results may not be reproducible in the current checkout; they remain evidence context, not a new execution claim.
- No Cardano Preview transaction E2E success is asserted. A valid Blockfrost credential, funded wallet, network access, and a controlled verification run are still required.
- JSON persistence is demo persistence, not a durable transactional store.
- A server-held single demo key is not an institutional custody design.
- The current service is Preview-only and is not a production or electoral certification boundary.

9. Recommendations for Pilot or Production

1. Define the legal, governance, and electoral requirements before treating any ledger event as authoritative.
2. Replace the demo key with Vault/HSM-backed custody, rotation, access controls, and preferably institutional multisignature approval.
3. Replace JSON persistence with durable transactional storage, migration, backup, and reconciliation procedures.
4. Add authenticated service-to-service identity, authorization, rate limits, retries, observability, alerting, and audit controls.
5. Threat-model commitment correlation, metadata leakage, replay, duplicate events, webhook authenticity, and key compromise.
6. Run a reproducible Preview test matrix, then a separately governed pilot before considering Mainnet or production use.
7. Increase unit, integration, failure-path, privacy, and operational coverage; publish metrics with test commands and environment details.

10. Public References

- Hyperledger Identus documentation
- Cardano testnet overview
- Cardano developer portal
- Blockfrost documentation
- Fastify documentation
- Lucid Evolution repository
- .NET documentation